Специальность: 5-04-0612-02 «Разработка и сопровождение программного обеспечения информационных систем»	Учебная дисциплина: «Основы кроссплатформенного программирования»

Лабораторная работа № 5
Инструкционно-технологическая карта

Тема: Разработка, отладка и испытание программ с использованием классов и созданием объектов

Цель: Научиться создавать классы, методы и экземпляры классов, а также обращаться к ним в языке Java

Время выполнения: 4 часа

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Изучить теоретические сведения;
2. Ответить на контрольные вопросы;
3. Откомпилировать примеры программ из раздела «Теоретические сведения»;
4. Выполнить ИДЗ.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ДЛЯ ВЫПОЛНЕНИЯ РАБОТЫ

В этом лабораторной работе будет рассмотрено создание класса на языке java и последующее обращение к этому классу, то есть мы научимся основам работы с классами.

Создание класса в java

Создавать класс мы будем в программе IntellJ IDEA, но также можно работать в любой другой IDE или в блокноте. В качестве примера мы будем создавать класс, который описывает характеристики и поведение кота.

Создадим в нашем проекте новый класс под именем Cat. Для этого в меню выберем File -> new -> class и, в открывшемся окне, введем имя класса – Cat. В итоге, у нас создался новый класс.

Структура класса в java

Класс в java оформляется с помощью следующей конструкции:

```
public class Cat { }
```

Рассмотрим ее составляющие:

public – модификатор доступа к классу, в данном случае он нам говорит, что этот класс будет доступен не только данному классу, но и другим. В java существуют и другие модификаторы, но об этого поговорим позднее. class – ключевое слово, говорящее о том, что это класс.

Cat– имя класса. Имена классов принято писать с заглавной буквы. { } – фигурные скобки, между которыми разместится тело нашего класса.

Классы в java состоят из атрибутов и методов, присвоим некоторые и нашему коту.

Атрибуты класса Cat

Атрибутами кота могут быть: имя, вес, окраска. Атрибуты – это переменные, которые объявляются следующим образом:

```
private int weight; // вес кота
private String name; // имя кота
private String color; //окрас кота
```

private – здесь опять же указывает на права доступа. К переменной, в данном случае, можно обращаться только из ее класса, чтобы классы извне не могли ее изменить.

int, String – это типы данных. В данном случае, вес будет задан при помощи целого числа – int, а имя и цвет при помощи символьной строки String. После объявления каждого атрибута должна ставиться точка с запятой; При помощи // в коде программы оформляются комментарии, которые являются правилом хорошего тона.

Методы класса Cat

Пусть наш кот умеет есть, спать и разговаривать. Опишем это поведение с помощью методов.

```
//кот ест public void eat(){ System.out.print(“Eating...\n”); }
```

public void eat()– сигнатура метода, его определение. В данном случае она нам говорит о том, что:

- метод eat() доступен для других классов – public;
- метод eat() не возвращает никаких данных – void.

eat – имя метода. Имена методов, в отличие от имен классов, принято писать с маленькой буквы. На конце каждого метода после имени всегда идут круглые скобки ( ), пустые или содержащие параметры (об этом позднее).

После сигнатуры метода идут фигурные скобки { }.В них содержится тело метода. Тело нашего метода содержит просто вывод сообщения о том, что кот кушает – “Eating...\n”. За вывод текстового сообщения отвечает метод System.out.print(“Eating...\n”).

\n – символ перевода строки, чтобы при выводе наших сообщений все они не писались в одну строку.

Аналогично предыдущему методу опишем спящего кота:

```
//кот спит
 public void sleep(){
System.out.print(“Sleeping zz-z-z-z...\n”); }
```

И немного иначе опишем метод – кот говорит:

```
//кот говорит
public String speak(String words){
String phrase = words + “...mauu...\n”; return phrase;}
```

В отличие от предыдущих методов, этот метод возвращает значение и имеет входные параметры. Давайте подробнее рассмотрим сигнатуру метода public String speak(String words):

public- метод speak() доступен для других классов;

String -тип значения, которое возвращает метод. В предыдущих случаях ключевое слово void указывало на то, что метод ничего не возвращает. В данном случае String указывает на то, что метод возвращает значение типа строка.

В процессе своей работы, метод выполняет определенные действия над данными, иногда необходимо, чтобы результат этих действий был передан для дальнейшей обработки другим классам, в этом случае метод передает (возвращает) этот результат. Эти возвращаемые данные, относятся к какому либо типу, в нашем примере это тип символьной строки - String.

Возвращающие методы должны содержать в своем теле ключевое слово return, которое указывает на то, что именно возвращает данный метод. В нашем случае это переменная phrase.

speak- имя метода.

(String words) – входные параметры. Входные параметры, это какие-либо данные, которые передаются из других классов и, которые метод должен обработать. Наш метод получает в качестве входных данных строку в виде переменной words, к этой строке дописывает «...mauu...» и возвращает то, что получилось.

В итоге класс Cat выглядит следующим образом:

```
public class Cat {
private int weight; // вес кота
private String name; // имя кота
private String color; //окрас кота
//кот ест
 public void eat(){
System.out.print(“Eating...\n”); }
//кот спит
public void sleep(){
System.out.print(“Sleeping zz-z-z-z...\n”); }
//кот говорит
public String speak(String words){
 String phrase = words + “...mauu...\n”; return phrase; } }
```

Обращение к классу в Java

Далее рассмотрим обращение к классу Cat. И то, как его использовать в работе.

Создадим класс Main: File -> new -> class, ввести в открывшемся окне имя класса.

public static void main(String[] args) – сигнатура метода main(). Программа состоит из нескольких классов, но только один из классов может содержать метод main(). Метод main(), это так называемая точка входа в программу. Без этого метода мы не сможем стартовать программу. Теперь, вместо приветствия

миру, этот метод будет содержать код для работы с классом Cat.

Пишем в тело метода main() (между его фигурными скобками) следующие строки:

```
Cat ourcat = new Cat();
```



```
ourcat.eat();
ourcat.sleep();
String say = ourcat.speak(“Play with me”);
System.out.println(say);
```

Разберем подробнее, что это значит.

Прежде чем вызывать, созданные нами методы в классе Cat, и заставить нашего кота есть, спать и говорить, сперва нужно создать экземпляр класса(инстанцию).

```
Cat ourcat = new Cat();
```

Данная строчка нам говорит о том, что в памяти создан экземпляр объекта Cat, а переменная ourcat типа Cat (такого же, как и наш объект) указывает на то место в памяти, где был этот объект создан.

Переменную ourcat теперь можно использовать для вызова методов класса Cat, например:

```
ourcat.eat(); ourcat.sleep();
```

Если метод возвращает какое-либо значение, например, как наш метод speak()возвращает значение типа String, то его можно вызывать следующим образом:

- объявить переменную такого же типа, что и возвращаемое значение (в нашем случае String)
- присвоить ей вызванный метод, например:

```
String say = ourcat.speak(“Play with me”);
```

Вспомним, что при описании нашего метода он содержал параметры speak(String words). Теперь, при вызове в качестве параметра выступила фраза “Play with me”, метод speak() ее обработал и вернул “Play with me...mauu...”. Именно это значение он присвоил переменной say.

Мы это можем проверить, выведя say на печать при помощи команды:

```
System.out.println(say);
```

Итак, наш класс Hello теперь выглядит следующим образом:

```
public class Hello {
 / * @param args */
public static void main(String[] args) {
Cat ourcat= new Cat();
ourcat.eat();
ourcat.sleep();
String say = ourcat.speak(“Play with me”);
System.out.println(say); } }
```

Теперь сохраним и запустим нашу программу.

В качестве результата внизу в консоле мы получаем следующие строки.

Eating...Sleeping zz-z-z-z...Play with me...mauu...

Перечисления

При разработке приложений достаточно часто встречаются сущности, число значений которых ограничено естественным образом: страны мира, месяцы года, дни недели, марки транспортных средств, типы пользователей и многие другие. В этих случаях до Java 5 прибегали к следующему решению:


```
package by.epam.learn.type;
public class RoleOldStyle {

public final static int GUEST = 0;
public final static int CLIENT = 1;
public final static int MODERATOR = 2;
public final static int ADMIN = 3;
private RoleOldStyle(){}
}
```

Обрабатывались такие значения следующим образом:

```
package by.epam.learn.type;
public class RoleOldStyleMain {
public static void main(String[] args) {
int role = 1;
switch (role){
case 0:
System.out.println("guest can only watch");
case 1:
System.out.println("client can place orders");
case 2:
System.out.println("moderator can manage his section");
case 3:
System.out.println("admin controls everything");
}
}
}
```

Не очень очевидное решение. Но Java 5 был введен класс-перечисление, и тогда в данной задаче код становится более читаемым:

```
package by.epam.learn.type;
public enum Role {
GUEST, CLIENT, MODERATOR, ADMIN
}
package by.epam.learn.type;
public class RoleMain {
public static void main(String[] args) {
Role role = Role.valueOf("Admin".toUpperCase());
switch (role){
case GUEST:
```

```
System.out.println(role + " can only watch");
case CLIENT:
System.out.println(role + " can place orders");
case MODERATOR:
System.out.println(role + " can manage his section");
}
```



```
case ADMIN:
System.out.println(role + " controls everything");
}
}
}
```

Типобезопасные перечисления (typesafe enums) в Java представляют собой классы и являются подклассами абстрактного класса java.lang.Enum. Вместо слова class при описании перечисления используется слово enum. При этом объекты перечисления инициализируются прямым объявлением без помощи оператора new. При инициализации хотя бы одного перечисления происходит инициализация всех без исключения оставшихся элементов данного перечисления.

В операторах case используются константы без уточнения типа перечисления, так как его тип определен в switch.

Перечисление как подкласс класса Enum может содержать поля, конструкторы и методы, реализовывать интерфейсы. Каждый элемент enum может использовать методы:

static enumType[] values() – возвращает массив, содержащий все элементы перечисления в порядке их объявления;

static <T extends Enum<T>> T valueOf(Class<T> enumType, String arg) – создает элемент перечисления, соответствующий заданному типу и значению передаваемой строки;

static enumType valueOf(String arg) – создает элемент перечисления, соответствующий значению передаваемой строки;

int ordinal() – возвращает позицию элемента перечисления, начиная с нуля, следствием чего является возможность сравнения элементов перечисления между собой на больше\меньше соответствующими операторами;

String name() – возвращает имя элемента, так же как и toString(); int compareTo(T t) – сравнивает элементы на больше, меньше либо равно.

Пример создания объекта:

```
Role role = Role.valueOf("client".toUpperCase());
```

Класс перечисления может объявлять собственные методы, и, следовательно, экземпляры перечисления могут к этим методам обращаться. Перечисления представляют собой классы, а классам положено явно или неявно объявлять конструкторы.

Перечисление является классом, поэтому в его теле можно объявлять,

кроме методов, также поля и конструкторы. Все конструкторы вызываются автоматически при инициализации любого из элементов. Конструктор не может быть объявлен со спецификаторами public и protected, так как не вызывается явно и перечисление не может быть суперклассом. Поля перечисления используются для сохранения дополнительной информации, связанной с его элементом.

Метод toString() реализован в классе Enum для вывода элемента в виде строки. Если переопределить метод toString() в конкретной реализации перечисления, то можно выводить не только значение элемента, но и значения его полей, то есть предоставить полную информацию об объекте, как и определяется контрактом метода.

Метод ordinal() определяет порядковый номер элемента перечисления, обратная задача не совсем тривиальна, но решаема. Определение элемента перечисления по позиции с применением метода values() преобразования элементов перечисления в массив:

```
public static Optional<Role> valueOf(int roleId) {
return Arrays.stream(values())
.filter(roleType -> roleType.ordinal() == roleId)
.findFirst();
}
```

При разработке класса перечисления возможны различные ошибки. В частности:

```
enum WrongPlanType {
AIRBUS_A320, AIRBUS_A380, AIRBUS_A330, BOEING_737_800, BOEING_777
}
class WrongPlane {
private WrongPlanType type;
}
```

Для описания типов и марок самолетов создано перечесление, но такой класс в итоге может оказаться очень большим из-за количества моделей конкретного производителя. Решить проблему легко разделением ответственности за хранение типа производителя в перечислении, а марки модели в обычной строке:

```
enum PlanType {
AIRBUS, BOEING
}
class Plane {
private PlanType type;
private String model;
}
```

На перечисления накладывается целый ряд ограничений. Им запрещено:

• быть суперклассами;

• быть подклассами;
• быть абстрактными;
• быть параметризированными;
• создавать экземпляры, используя ключевое слово new.

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Структура класса в Java.
2. Как создаются методы класса?
3. Каким образом создается экземпляр класса?
4. Синтаксис класса.
5. Что такое перечисление и как запсывается

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Задание 1.

1. Класс “Студент” (Student): Поля: “name” (String), “age” (int), “group” (String). Конструктор (все поля). Метод “displayInfo()”. В “main” создать 2 объекта и вывести их данные.

2. Класс “Книга” (Book): Поля: “title”, “author”, “year” (int). Конструктор. Метод “isOld()” (возвращает true, если книга старше 50 лет). В “main” создать 3 книги и вывести только старые.

3. Класс “Прямоугольник” (Rectangle): Поля: “width”, “height” (double). Конструктор. Методы: “area()”, “perimeter()”. В “main” создать несколько прямоугольников и вывести их параметры.

4. Класс “Точка” (Point): Поля: “x”, “y” (double). Конструктор. Метод “distanceTo(Point other)” (расстояние до другой точки). В “main” создать две точки и вывести расстояние.

5. Класс “Дробь” (Fraction): Поля: “numerator”, “denominator” (int). Конструктор. Методы: “toDecimal()” (double), “toString()” (в формате “числитель/знаменатель”). Создать дроби 1/2, 3/4 и вывести их.

6. Класс “Автомобиль” (Car): Поля: “brand”, “model”, “year”, “mileage” (int). Конструктор. Метод “drive(int km)” (увеличивает пробег). Метод “displayInfo()”. В “main” создать машину, “проехать” 100 км и вывести обновленный пробег.

7. Класс “Окружность” (Circle): Поля: “radius” (double). Конструктор. Методы: “area()”, “circumference()”. В “main” создать несколько окружностей и вывести их параметры.

8. Класс “Треугольник” (Triangle): Поля: “sideA”, “sideB”, “sideC” (double). Конструктор. Метод “isValid()” (проверяет существование треугольника). Метод “perimeter()”. В “main” создать треугольник и вывести периметр или сообщение об ошибке.

9. Класс “Сотрудник” (Employee): Поля: “name”, “position”, “salary” (double). Конструктор. Метод “raiseSalary(double percent)”. В “main” создать

сотрудника, повысить зарплату на 10% и вывести новую зарплату.

10. Класс “Телефон” (Phone): Поля: “brand”, “model”, “price” (double). Конструктор. Метод “applyDiscount(double discount)” (уменьшает цену). В “main” создать телефон, применить скидку 15%, вывести новую цену.

11. Класс “Квартира” (Apartment): Поля: “address”, “rooms” (int), “area” (double). Конструктор. Метод “pricePerSquareMeter(double totalPrice)”. В “main” создать квартиру и вывести цену за квадратный метр.

12. Класс “Товар” (Product): Поля: “name”, “price” (double), “quantity” (int). Конструктор. Метод “totalCost()” (возвращает price * quantity). В “main” создать товар и вывести общую стоимость.

13. Класс “Поезд” (Train): Поля: “destination”, “trainNumber”, “departureTime” (String). Конструктор. Метод “isDelayed(int minutes)” (изменяет время отправления). В “main” создать поезд и задержать его на 30 минут.

14. Класс “Собака” (Dog): Поля: “name”, “breed”, “age” (int). Конструктор. Метод “bark()” (выводит “Гав!”). В “main” создать собаку и заставить ее лаять.

15. Класс “Счетчик” (Counter): Поля: “count” (int). Конструктор (устанавливает 0). Методы: “increment()”, “decrement()”, “getValue()”. В “main” создать счетчик, увеличить его 5 раз, уменьшить 2 раза, вывести значение.

16. Класс “Круглый стол” (RoundTable): Поля: “diameter” (double), “material”. Конструктор. Метод “area()”. В “main” создать стол и вывести его площадь.

17. Класс “Ноутбук” (Laptop): Поля: “brand”, “ram” (int), “storage” (int), “price”. Конструктор. Метод “applyDiscount()”. В “main” создать ноутбук и применить скидку.

18. Класс “Билет” (Ticket): Поля: “event”, “seatNumber”, “price”. Конструктор. Метод “applyDiscount()”. В “main” создать билет и применить скидку.

19. Класс “Здание” (Building): Поля: “address”, “floors” (int), “totalArea”. Конструктор. Метод “averageAreaPerFloor()”. В “main” создать здание и вывести среднюю площадь этажа.

20. Класс “Человек” (Person): Поля: “firstName”, “lastName”, “birthYear”. Конструктор. Метод “getAge()” (вычисляет возраст). В “main” создать человека и вывести его возраст.

21. Класс “Магазин” (Store): Поля: “name”, “address”, “employeeCount”. Конструктор. Метод “isLarge()” (true, если сотрудников > 50). В “main” создать магазин и проверить.

22. Класс “Ресторан” (Restaurant): Поля: “name”, “cuisine”, “rating” (int). Конструктор. Метод “isPopular()” (true, если рейтинг > 4). В “main” создать ресторан и проверить.

23. Класс “Фильм” (Movie): Поля: “title”, “director”, “year”, “rating”. Конструктор. Метод “isExcellent()” (true, если рейтинг >= 8). В “main” создать фильм и проверить.

24. Класс “Смартфон” (Smartphone): Поля: “brand”, “screenSize” (double), “batteryLife” (int). Конструктор. Метод “isGoodBattery()” (true, если > 10 часов). В “main” создать смартфон и проверить.

25. Класс “Игра” (Game): Поля: “title”, “genre”, “rating” (int). Конструктор. Метод “isTop()” (true, если рейтинг >= 9). В “main” создать игру и проверить.

26. Класс “Билет” (Ticket): Поля: “event”, “seatNumber”, “price”. Конструктор. Метод “applyDiscount()”. В “main” создать билет и применить скидку.

27. Класс “Здание” (Building): Поля: “address”, “floors” (int), “totalArea”. Конструктор. Метод “averageAreaPerFloor()”. В “main” создать здание и вывести среднюю площадь этажа.

28. Класс “Человек” (Person): Поля: “firstName”, “lastName”, “birthYear”. Конструктор. Метод “getAge()” (вычисляет возраст). В “main” создать человека и вывести его возраст.

29. Класс “Товар” (Product): Поля: “name”, “price” (double), “quantity” (int). Конструктор. Метод “totalCost()” (возвращает price * quantity). В “main” создать товар и вывести общую стоимость.

30. Класс “Поезд” (Train): Поля: “destination”, “trainNumber”, “departureTime” (String). Конструктор. Метод “isDelayed(int minutes)” (изменяет время отправления). В “main” создать поезд и задержать его на 30 минут.

Задание 2.

1. Класс "Товар" (name: String, price: double, quantity: int). В “main” создать массив из 5 товаров. Найти товар с максимальной ценой и общую стоимость всех товаров (price * quantity).

2. Класс "Студент" (name: String, grade: double, group: String). Создать массив из 7 студентов. Найти студента с максимальной оценкой и среднюю оценку всех студентов.

3. Класс "Книга" (title: String, author: String, year: int). Создать массив из 5 книг. Найти самую старую книгу и самую новую книгу.

4. Класс "Сотрудник" (name: String, salary: double, position: String). Создать массив из 6 сотрудников. Найти сотрудника с максимальной зарплатой и сотрудника с минимальной зарплатой.

5. Класс "Автомобиль" (brand: String, year: int, price: double). Создать массив из 5 автомобилей. Найти самое дорогое авто и самое старое авто. 6. Класс "Фильм" (title: String, director: String, rating: double). Создать массив из 6 фильмов. Найти фильм с максимальным рейтингом и средний рейтинг всех фильмов.

7. Класс "Игра" (title: String, genre: String, price: double). Создать массив из 5 игр. Найти самую дорогую игру и самую дешевую игру. 8. Класс "Квартира" (address: String, rooms: int, price: double). Создать массив из 6 квартир. Найти квартиру с максимальным количеством комнат и самую дорогую квартиру.

9. Класс "Смартфон" (brand: String, price: double, batteryLife: int). Создать массив из 5 смартфонов. Найти смартфон с максимальной емкостью батареи и среднюю цену всех смартфонов.

10. Класс "Ноутбук" (brand: String, ram: int, price: double). Создать массив из 6 ноутбуков. Найти ноутбук с максимальным объемом RAM и самый дорогой ноутбук.

11. Класс "Ресторан" (name: String, cuisine: String, rating: double). Создать массив из 5 ресторанов. Найти ресторан с максимальным рейтингом и средний рейтинг всех ресторанов.

12. Класс "Отель" (name: String, stars: int, pricePerNight: double). Создать массив из 6 отелей. Найти отель с максимальным количеством звезд и самый дорогой отель.

13. Класс "Спортсмен" (name: String, sport: String, score: double). Создать массив из 5 спортсменов. Найти спортсмена с максимальным результатом и средний результат всех спортсменов.

14. Класс "Город" (name: String, population: int, area: double). Создать массив из 6 городов. Найти город с максимальным населением и город с максимальной площадью.

15. Класс "Страна" (name: String, population: int, capital: String). Создать массив из 5 стран. Найти страну с максимальным населением и вывести все столицы.

16. Класс "Планета" (name: String, diameter: double, distanceFromSun: double). Создать массив из 6 планет. Найти планету с максимальным диаметром и планету с максимальным расстоянием от Солнца.

17. Класс "Поезд" (destination: String, trainNumber: int, travelTime: int). Создать массив из 5 поездов. Найти поезд с максимальным временем в пути и поезд с минимальным временем в пути.

18. Класс "Самолет" (model: String, capacity: int, range: int). Создать массив из 6 самолетов. Найти самолет с максимальной вместимостью и самолет с максимальной дальностью полета.

19. Класс "Корабль" (name: String, length: double, tonnage: double). Создать массив из 5 кораблей. Найти самый длинный корабль и корабль с максимальным водоизмещением.

20. Класс "Здание" (address: String, floors: int, totalArea: double). Создать массив из 6 зданий. Найти здание с максимальным количеством этажей и здание с максимальной площадью.

21. Класс "Школа" (name: String, students: int, teachers: int). Создать массив из 5 школ. Найти школу с максимальным количеством учеников и школу с максимальным количеством учителей.

22. Класс "Больница" (name: String, beds: int, doctors: int). Создать массив из 6 больниц. Найти больницу с максимальным количеством коек и больницу с максимальным количеством врачей.

23. Класс "Магазин" (name: String, area: double, employees: int). Создать массив из 5 магазинов. Найти магазин с максимальной площадью и магазин с максимальным количеством сотрудников.

24. Класс "Парк" (name: String, area: double, trees: int). Создать массив из 6 парков. Найти парк с максимальным количеством деревьев и парк с максимальной площадью.

25. Класс "Озеро" (name: String, depth: double, area: double). Создать массив из 5 озер. Найти самое глубокое озеро и озеро с максимальной площадью.

26. Класс "Гора" (name: String, height: double, country: String). Создать массив из 6 гор. Найти самую высокую гору и среднюю высоту всех гор. 27. Класс "Река" (name: String, length: double, source: String). Создать массив из 5 рек. Найти самую длинную реку и вывести все истоки. 28. Класс "Музей" (name: String, exhibitions: int, visitors: int). Создать массив из 6 музеев. Найти музей с максимальным количеством экспонатов и музей с максимальным количеством посетителей.

29. Класс "Театр" (name: String, seats: int, plays: int). Создать массив из 5 театров. Найти театр с максимальным количеством мест и театр с максимальным количеством спектаклей.

30. Класс "Спортивная команда" (name: String, players: int, wins: int). Создать массив из 6 команд. Найти команду с максимальным количеством игроков и команду с максимальным количеством побед.

Задание 3.

1. Enum (Season): Enum “Season” (WINTER, SPRING, SUMMER, AUTUMN). Класс “Месяц” (name, season). В “main” создать список месяцев и вывести летние.

2. Enum (DayOfWeek): Enum “DayOfWeek” с методом “isWeekend()”. В “main” вывести все дни недели с указанием, выходной это или рабочий. 3. Enum (Gender): Enum “Gender” (MALE, FEMALE). Класс “Человек” с полем gender. В “main” создать мужчину и женщину, вывести их. 4. Enum (Priority): Enum “Priority” (LOW, MEDIUM, HIGH). Класс “Задача” с полем priority. В “main” создать задачи разного приоритета и отсортировать по приоритету.

5. Enum (Role): Enum “Role” (ADMIN, USER, GUEST). Класс “Пользователь” с полем role. В “main” создать пользователей с разными ролями и вывести только админов.

6. Enum "Season" (WINTER, SPRING, SUMMER, AUTUMN). Класс "Месяц" (name: String, season: Season). В “main” создать список из 12 месяцев и вывести только зимние месяцы.

7. Enum "DayOfWeek" (MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY) с методом “isWeekend()”. В “main” вывести все дни недели с указанием, выходной это или рабочий.

8. Enum "Gender" (MALE, FEMALE). Класс "Человек" (name: String, gender: Gender). В “main” создать 3 мужчин и 2 женщин, вывести только

женщин.

9. Enum "Priority" (LOW, MEDIUM, HIGH). Класс "Задача" (description: String, priority: Priority). В “main” создать 5 задач с разным приоритетом и вывести задачи с приоритетом HIGH.

10. Enum "Role" (ADMIN, USER, GUEST). Класс "Пользователь" (username: String, role: Role). В “main” создать 5 пользователей и вывести только администраторов (ADMIN).

11. Enum "Color" (RED, GREEN, BLUE, YELLOW, BLACK, WHITE). Класс "Автомобиль" (brand: String, color: Color). В “main” создать 6 автомобилей и вывести все красные (RED) автомобили.

12. Enum "Genre" (FICTION, NON_FICTION, SCIENCE, HISTORY, FANTASY). Класс "Книга" (title: String, genre: Genre). В “main” создать 5 книг разных жанров и вывести все книги жанра FANTASY.

13. Enum "Category" (FOOD, CLOTHES, ELECTRONICS, BOOKS, TOYS). Класс "Товар" (name: String, category: Category). В “main” создать 6 товаров и вывести все товары категории ELECTRONICS.

14. Enum "Status" (NEW, IN_PROGRESS, COMPLETED, CANCELLED). Класс "Заказ" (orderNumber: int, status: Status). В “main” создать 5 заказов и вывести все заказы со статусом COMPLETED.

15. Enum "Type" (SEDAN, SUV, HATCHBACK, COUPE, CONVERTIBLE). Класс "Автомобиль" (model: String, type: Type). В “main” создать 6 автомобилей и вывести все автомобили типа SUV.

16. Enum "Education" (SCHOOL, BACHELOR, MASTER, PHD). Класс "Сотрудник" (name: String, education: Education). В “main” создать 5 сотрудников и вывести всех сотрудников с образованием PHD.

17. Enum "Department" (IT, HR, FINANCE, MARKETING, SALES). Класс "Сотрудник" (name: String, department: Department). В “main” создать 6 сотрудников и вывести всех сотрудников из отдела IT.

18. Enum "PaymentMethod" (CASH, CARD, CRYPTO, TRANSFER). Класс "Платеж" (amount: double, method: PaymentMethod). В “main” создать 5 платежей и вывести все платежи, совершенные картой (CARD).

19. Enum "TicketType" (ECONOMY, BUSINESS, FIRST_CLASS). Класс "Билет" (flightNumber: String, type: TicketType). В “main” создать 6 билетов и вывести все билеты бизнес-класса (BUSINESS).

20. Enum "FuelType" (PETROL, DIESEL, ELECTRIC, HYBRID). Класс "Автомобиль" (model: String, fuelType: FuelType). В “main” создать 5 автомобилей и вывести все электромобили (ELECTRIC).

21. Enum "InsuranceType" (HEALTH, LIFE, CAR, PROPERTY). Класс "Страховка" (number: String, type: InsuranceType). В “main” создать 6 страховок и вывести все страховки типа HEALTH.

22. Enum "Subscription" (FREE, PREMIUM, VIP). Класс "Пользователь" (username: String, subscription: Subscription). В “main” создать 5 пользователей

и вывести всех пользователей с подпиской VIP.

23. Enum "ExamResult" (PASS, FAIL, EXCELLENT). Класс "Экзамен" (subject: String, result: ExamResult). В “main” создать 6 экзаменов и вывести все экзамены с результатом EXCELLENT.

24. Enum "OrderStatus" (PENDING, SHIPPED, DELIVERED, RETURNED). Класс "Заказ" (id: int, status: OrderStatus). В “main” создать 5 заказов и вывести все заказы со статусом SHIPPED.

25. Enum "Transport" (BUS, TRAIN, PLANE, SHIP, CAR). Класс "Поездка" (destination: String, transport: Transport). В “main” создать 6 поездок и вывести все поездки на поезде (TRAIN).

26. Enum "Language" (RUSSIAN, ENGLISH, FRENCH, GERMAN, SPANISH). Класс "Книга" (title: String, language: Language). В “main” создать 5 книг и вывести все книги на английском языке (ENGLISH).

27. Enum "Season" (WINTER, SPRING, SUMMER, AUTUMN). Класс "Мероприятие" (name: String, season: Season). В “main” создать 6 мероприятий и вывести все мероприятия, проходящие летом (SUMMER).

28. Enum "MaritalStatus" (SINGLE, MARRIED, DIVORCED, WIDOWED). Класс "Человек" (name: String, status: MaritalStatus). В “main” создать 5 человек и вывести всех женатых/замужних (MARRIED).

29. Enum "DrinkSize" (SMALL, MEDIUM, LARGE, XL). Класс "Напиток" (name: String, size: DrinkSize). В “main” создать 6 напитков и вывести все напитки размера XL.

30. Enum "Weekend" (WORKDAY, WEEKEND). Класс "День" (date: String, type: Weekend). В “main” создать 7 дней (понедельник-воскресенье) и вывести только выходные дни (WEEKEND).

ДОМАШНЕЕ ЗАДАНИЕ

Индивидуальное задание

ЛИТЕРАТУРА

Блинов, И. Н., Романчик, В. С. Java from EPAM : учеб.-метод. пособие / И. Н. Блинов, В. С. Романчик. Минск: Четыре четверти, 2021. 560 с.

Преподаватель А.С.Кибисова